

独立文档 · 实现参考

MTV 开发者实现参考

客户端-服务端分离架构、Channel 核心模型、时间轴与网络包同步流程。

McediaTV

文档版 1.0 · 2026.06

基于当前项目实现整理

| c/s 分离架构

MTV 采用客户端 Mod(运行在玩家本地)与服务端插件(运行在 Minecraft 服务端)分离的架构。两者通过自定义网络包通信。

服务端插件职责

- 维护 Channel 的权威状态
- 管理 Audience 加入、心跳、超时与退出
- 维护播放器与频道的绑定关系
- 播放列表推进逻辑
- 向客户端广播状态更新
- Channel 状态持久化

客户端 Mod 职责

- 维护多个播放器外设实例
- 将 Screen/Speaker 外设挂接到指定 Channel
- 接收服务端状态并执行媒体加载、播放、暂停、跳转
- 周期性发送心跳附带本地观测状态
- 媒体加载完成后上报元信息

客户端是控制发起者，但不是权威状态写入者。所有"设置链接、播放、暂停、快进、调速"等操作，都必须先作为请求发送到服务端，由服务端修改 Channel 状态，再通过状态同步广播给所有受众。客户端收到更新后才执行本地播放器调整。

核心原则：服务端是权威状态写入者。客户端播放器只是 snapshot 的执行者。客户端 heartbeat 只负责观测与反馈，不直接写权威状态。该链路追求的是"状态一致 + 5 秒级收敛"，不是逐帧严格同步。

Channel 核心模型

Channel 是管理媒体播放进度、播放状态、播放列表与 Audience 会话的核心对象。它不负责实际视频解码或音频播放，也不持有客户端播放器实体实例。

两种 Channel 类型

类型	绑定限制	可搜索	主要用途
Broadcast Channel (公共频道)	多个播放器可绑定	是	多屏同播，公共展示
Private Player Channel (私有 self 频道)	仅一个播放器绑定	否	独立播放，默认频道

私有频道不是临时 fallback 对象，而是 Channel 体系中的一等成员，同样需要持久化。

关键组件

服务端组件	职责
MtvChannelService	维护频道权威状态，负责持久化与触发 snapshot 广播。
MtvChannelNetworkService	处理 subscribe/unsubscribe/heartbeat，向客户端发送 snapshot/sync/remove。
AudienceSessionManager	维护 channel→audience 映射，active audience 过滤与多数派统计。
MtvPlaybackController	承接播放控制命令，转发给 MtvChannelService。
EntityPlayerHandle	每 tick 读取实体配置，解析目标 channelId，挂接或切换外设。

客户端组件	职责
ClientChannelPlaybackManager	管理 channelId→session 映射，处理 snapshot/sync/remove，决定何时 subscribe/unsubscribe。
ClientChannelSession	执行本地加载、seek、pause、speed 同步，周期性发送 heartbeat。

服务端权威时间轴

播放进度以服务端为权威，但服务端不会逐 tick 主动推进当前时间。它只保存权威播放核心状态与时间轴锚点。

核心字段

一个运行中的 Channel 至少需要维护：channelId、channelType、playState（包含 mediaUrl、state、speed、mediaTime、playTime）、duration、playlistCursor、playlist 和 revision。

播放位置推导公式

```
if playState.state == PLAYING:
    currentMediaTime = (currentSystemTime - playState.playTime)
                        * playState.speed + playState.mediaTime
else:
    currentMediaTime = playState.mediaTime
```

所有影响时间轴的操作都遵循同一规则：先根据当前锚点结算最新位置，再更新目标状态，重建新的时间轴锚点。

关键操作的时间轴语义

操作	时间轴更新逻辑
开始/恢复播放	state=PLAYING, mediaTime=当前位置, playTime=serverNow
暂停	先结算 currentMediaTime → mediaTime=currentMediaTime, state=PAUSED, playTime=serverNow
跳转	mediaTime=目标时间, playTime=serverNow, state 保持
调速	先结算 → mediaTime=currentMediaTime, speed=newRate, playTime=serverNow
切下一首	mediaUrl 切换, mediaTime=0, playTime=serverNow, 重置 duration, playlistCursor 前进

网络包与同步流程

C2S 包

包	发送时机	载荷
channel_subscribe	客户端对某 channel 的引用计数从 0→1	channelId
channel_unsubscribe	引用计数从 1→0	channelId
channel_heartbeat	每 5 秒, session 仍有 attachment 且有媒体	channelId , revision , loaded , completed , durationUs , error

S2C 包

包	发送时机	作用
channel_snapshot	状态变更后、周期性广播(~5s)、新订阅后	软刷新, 纠偏用, 含时间轴锚点
channel_sync	新订阅首包、客户端上报 error 时	强校准, 触发更积极的 seek 对齐
channel_remove	服务端认为频道失效时	客户端置空对应 session 的 snapshot

订阅机制要点

subscribe/unsubscribe 不按"实体"维度发送, 而是按"整个客户端是否仍需要该 channel"发送。同一客户端内的多个实体可复用同一个 channelId, 避免重复订阅和误退订。

客户端位置同步

```
位置 = anchorMediaTimeUs + elapsedTimeMs * 1000
      + localElapsedAfterReceiveMs * 1000 * speed
```

普通 snapshot 使用软阈值纠偏(3 秒), channel_sync 或 revision 明显变化时触发强校准(1 秒)。

Audience 与持久化

Audience 生命周期

- 加入：客户端通过首次 `subscribe(channelId)` 进入频道的 audience 集合。服务端立即下发当前状态。
- 心跳：每 5 秒一次。承担保活和状态观测上报两个职责。
- 超时：30 秒未收到心跳 → 标记断联并移除。失去全部 audience 的频道停止服务但保留持久化数据。

多数派统计口径

当前 active audience 必须同时满足：未超时 + 仍订阅当前 channel。在此范围内统计 loaded 和 completed 的多数派：

```
matched > 0 && loaded * 2 > matched // 多数派已加载
matched > 0 && completed * 2 > matched // 多数派已播完
```

duration 汇总取 active sessions 中上报的最大值。

新媒体加载到正式播放的流程

1. 服务端切换 `mediaUrl` → `ChannelRuntimeState` 进入 `LOADING` → 广播 snapshot
2. 客户端收到 snapshot, 开始异步加载媒体, 上报 `loaded = false`
3. 客户端媒体 ready 后开始上报 `loaded = true` 与 `durationUs`
4. 服务端判断多数派已加载 → 状态切到 `PLAYING` → 再次广播
5. 客户端按新 snapshot 正式进入统一播放状态

持久化设计

系统定义统一的 `ChannelRepository` 抽象，不把业务逻辑绑定到某一种存储实现。未来可支持多种后端：

- filesystem
- sqlite
- postgres
- mongodb

需持久化：`channelId`、`channelType`、`mediaUrl`、`mediaTime`、`playTime`、`speed`、`state`、`duration`、播放列表、`playlistCursor`、绑定关系。

不持久化：audience 在线集合、心跳到达时间、客户端瞬时观测值、会话期临时缓存。

设计决策记录

- 停止服务 $\neq$ 删除频道。失去全部 audience 只停止服务，持久化数据仍保留。
- 快进、快退、拖进度条在协议层统一归一为 `Seek(targetMediaTime)` 命令。
- 调速必须先结算旧时间轴再切换新倍率，否则导致位置跳变。
- 未拿到 duration 时，服务端不能做基于时长的自动切歌判断。